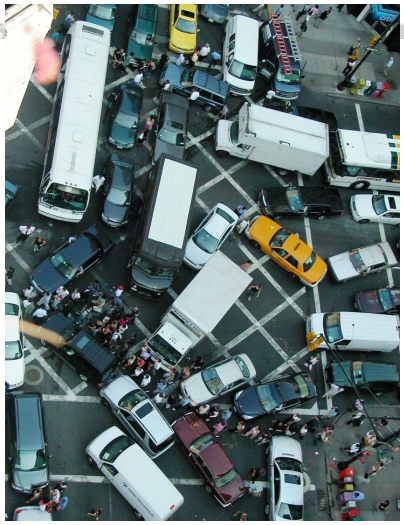


Deadlock

Michelle Kuttel

Department of Computer Science

University of Cape Town

Safety versus liveness

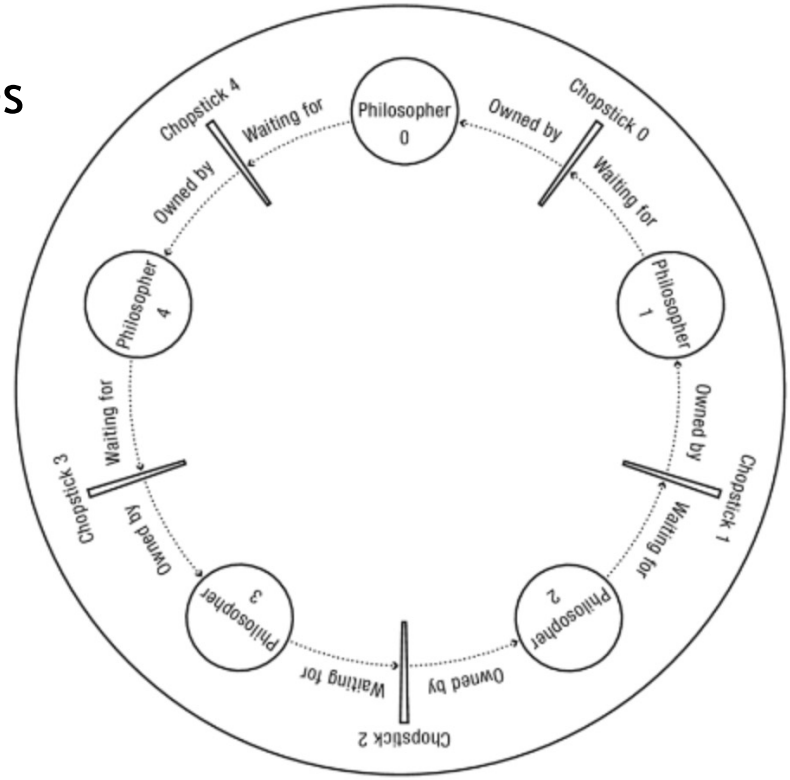
There are two types of correctness properties for concurrent programs:

Safety properties

The property must *always be true*.

Liveness properties

The property must eventually become true.



Solutions to **safety issues** often raise the possibility of **liveness issues.**

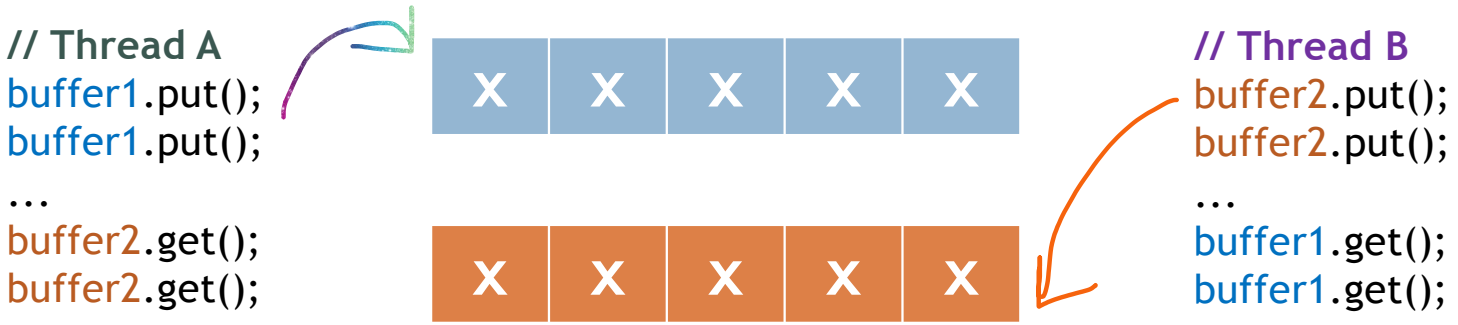
Deadlock in general

The problem of deadlock is *much broader* than just locks and condition variables.

- can occur anytime a thread **waits for an event that cannot happen** because of a cycle of waiting for a resource.
  - resources can be locks, or memory, or processing time, or disk blocks, or space in a buffer.

  - E.g if we have **two bounded buffers**, where
    - Thread A puts a request into buffer1 and then gets a response out of buffer2.
    - If **Thread B** does the reverse,

    Deadlock situation: Each thread may block trying to put into a full buffer while waiting for the other buffer to be drained.



Deadlock is a rare event

A system is *subject to deadlock* if a group of threads could deadlock under some circumstances.

- It may be fine for most runs deadlock only for particular workloads or “unlucky” interleavings.
- As testing may not discover deadlock problems, it is important to construct systems that are **deadlock-free by design**.



'*Cupids Fighting Putti* (c. 1625) by Guido Reni

System Model

System comprises:

- a **finite number** of **resources**
  - CPU cycles, files, and I/O devices
  - also synchronization tools such as **mutex locks** and **semaphores**
- distributed among a number of **competing threads**

System Model

System comprises:

- a **finite number** of resources
  - CPU cycles, files, and I/O devices
  - also synchronization tools such as **mutex locks** and **semaphores**
- distributed among a number of **competing** threads/processes

A thread/process may use a resource in only the following sequence:

1. **Request.**
2. **Use.**
3. **Release.**

System Model

System comprises:

- a **finite number** of **resources**
  - CPU cycles, files, and I/O devices
  - also synchronization tools such as **mutex locks** and **semaphores**
- distributed among a number of **competing threads**

A **set of threads** is in **deadlock** when **every thread** in the set **is waiting for an event** that can be **caused only by another thread in the set - so** none of those events can occur

Synchronization tools are the **most common sources of deadlock**.

Do Quiz 12

- Password: cycle

Deadlock Characterization

Four **necessary conditions** must be satisfied simultaneously for deadlock to occur:

- **Mutual exclusion**: only one process at a time can use a resource;
- **Hold and wait**: a process holding at least one resource is waiting to acquire additional resources held by other processes;
  - [sometimes called *multiple independent requests]*
- **No preemption**: a resource can be released only voluntarily by the process holding it, after that process has completed its task;
- **Circular wait**: there exists a set $\{P_0, P_1, \ldots, P_n\}$ of waiting processes such that P_0 is waiting for a resource that is held by P_1, P_1 is waiting for a resource that is held by P_2, …, P_{n-1} is waiting for a resource that is held by P_n, and P_n is waiting for a resource that is held by P_0.

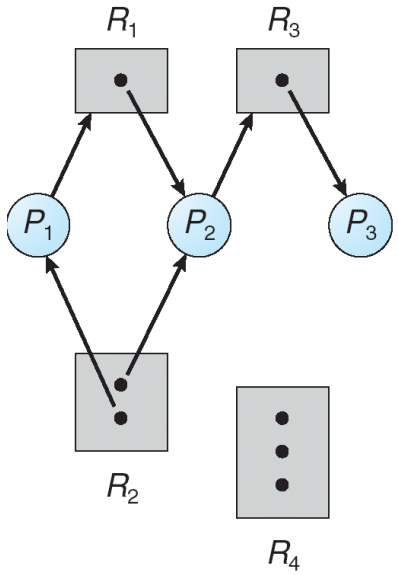


Deadlock Characterization: Resource-Allocation Graph

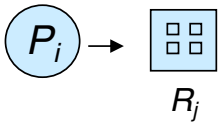
A set of vertices V and a set of edges E.

V is partitioned into two types:

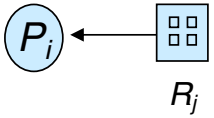
- $P = \{P_1, P_2, \ldots, P_n\}$, the set consisting of all the processes in the system
- $R = \{R_1, R_2, \ldots, R_m\}$, the set consisting of all resource types in the system



- **request edge** – directed edge $P_i \rightarrow R_j$



- **assignment edge** – directed edge $R_j \rightarrow P_i$



If graph contains **no cycles** $\Rightarrow$ **no deadlock**

If graph contains **a cycle** $\Rightarrow$

- If **only one instance** per resource type, then **deadlock exists**
- If **several instances** per resource type, then ***possibility*** **of deadlock**



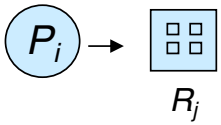
Deadlock Characterization: Resource-Allocation Graph

A set of vertices V and a set of edges E.

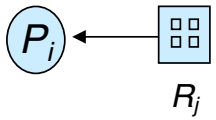
V is partitioned into two types:

- $P = \{P_1, P_2, \ldots, P_n\}$, the set consisting of all the processes in the system
- $R = \{R_1, R_2, \ldots, R_m\}$, the set consisting of all resource types in the system

- **request edge** – directed edge $P_i \rightarrow R_j$



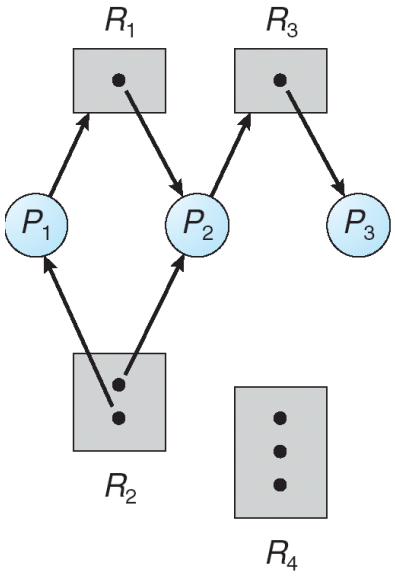
- **assignment edge** – directed edge $R_j \rightarrow P_i$



When thread requests an instance of resource, **a request edge** is inserted in the resource-allocation graph.

When this request can be fulfilled, the request edge is **instantaneously transformed to an assignment edge.**

When the thread no longer needs access to the resource, it releases the resource and assignment edge **is deleted.**

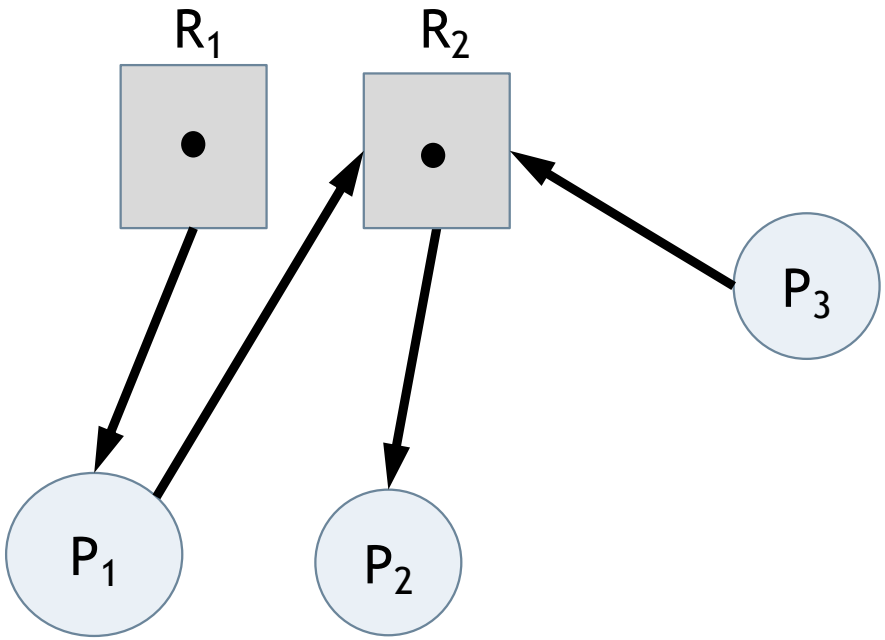


If graph contains **no cycles** ⇒ **no deadlock**

If graph contains **a cycle** ⇒

- If only one instance per resource type, then **deadlock exists**
- If several instances per resource type, then **possibility of deadlock**

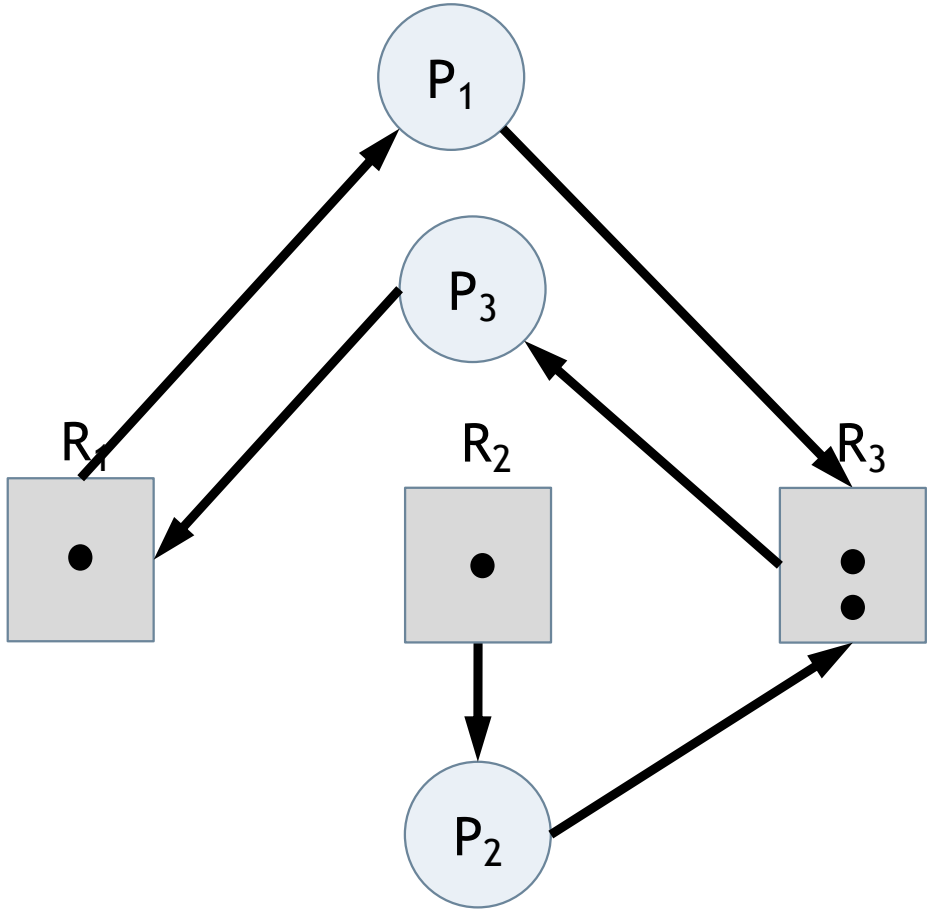
Checkpoint: Resource Allocation Graphs



Is there a deadlock here?



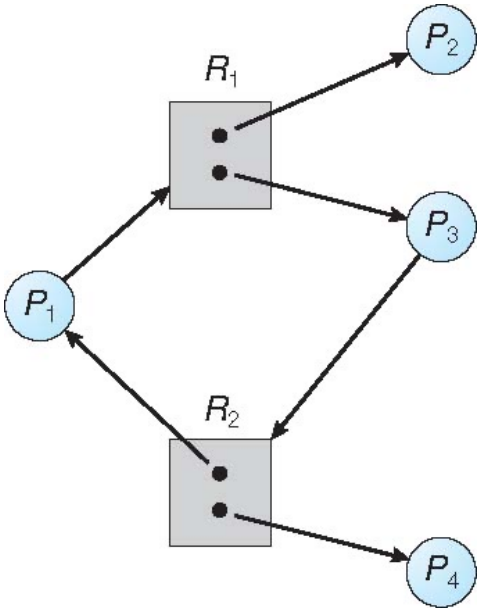
Checkpoint: Resource Allocation Graphs



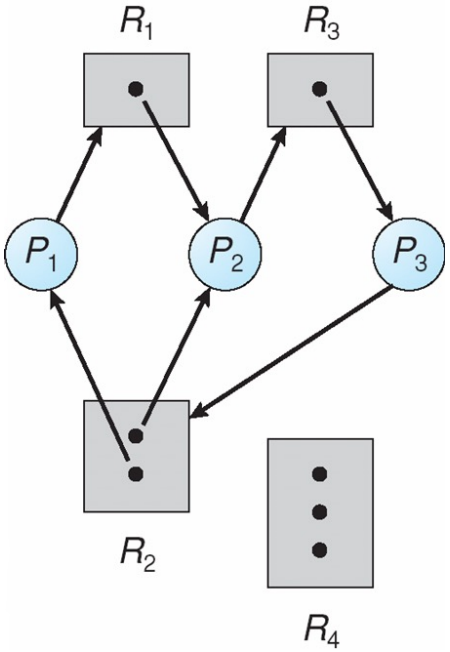
Is there a deadlock here?



Resource Allocation Graphs With a Cycle



A) Is there a deadlock?

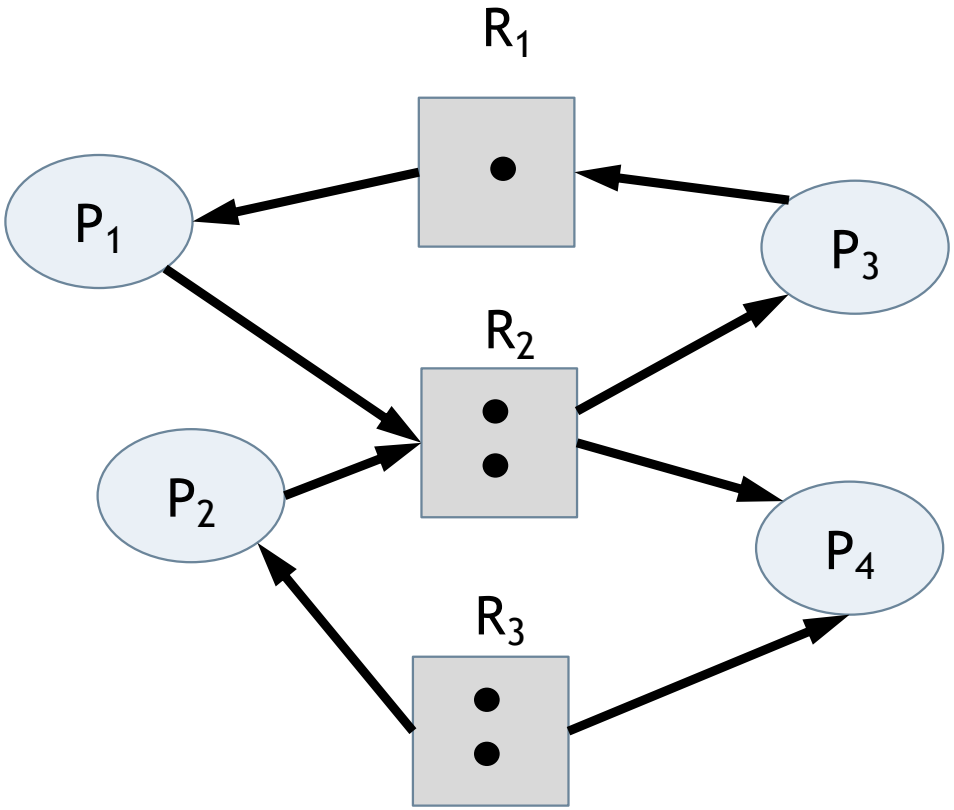


B) Is there a deadlock?

For those situations which are deadlock, provide the cycle of threads and resources that are deadlocked. Where there is no deadlock, provide an order in which threads may complete execution.



Checkpoint: Resource Allocation Graphs



Is there a deadlock?



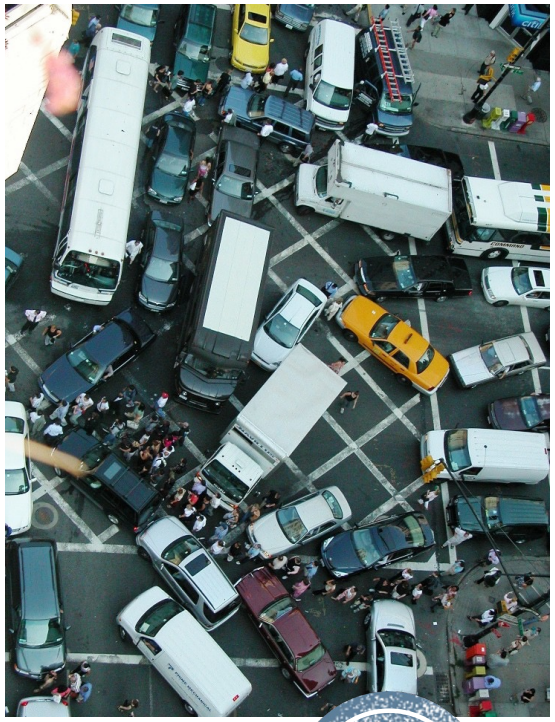
Handling Deadlock: Prevention &Avoidance

Michelle Kuttel

Department of Computer Science

University of Cape Town

June 2020



Methods for handling deadlock

- Pretend that deadlocks never occur in the system (ignore the problem).



  - **e.g most OS's**, including Windows and Linux!
    - up to **kernel and application developers** to **write programs that handle deadlocks**

- Use a **protocol to prevent or avoid deadlocks**, ensuring that the system will never enter a deadlocked state.



  - Deadlock **prevention**
    - ensure that at least **one of the necessary conditions cannot hold**



  - Deadlock **avoidance**
    - OS requires **advance information** on which resources a thread will request



- Allow the system to enter a deadlocked state, detect it, and recover
  - E.g. done in databases

Why do OS's not handle deadlock?

Undetected deadlock is bad:

will cause the system's performance to deteriorate, because resources are held by threads that cannot run…. and more and more threads, as they make requests for resources, will enter a deadlocked state.

Eventually, the system will stop functioning and will need to be **restarted manually**.

So, why don't OS's handle deadlock?

Methods for handling deadlock

- Pretend that deadlocks never occur in the system (ignore the problem).



  - **e.g most OS's**, including Windows and Linux!
    - up to **kernel and application developers** to **write programs that handle deadlocks**

- Use a **protocol to prevent or avoid deadlocks**, ensuring that the system will never enter a deadlocked state.
  - Deadlock **prevention**
    - ensure that at least **one of the necessary conditions cannot hold**
  - Deadlock **avoidance**
    - OS requires **advance information** on which resources a thread will request

- Allow the system to enter a deadlocked state, detect it, and recover
  - E.g. done in databases



Deadlock Prevention

Four necessary conditions must be satisfied **simultaneously** for deadlock to occur. Which of these can we remove?

- **Mutual exclusion**: only one process at a time can use a resource;
- **Hold and wait**: a process holding at least one resource is waiting to acquire additional resources held by other processes;
- **No preemption**: a resource can be released only voluntarily by the process holding it, after that process has completed its task;
- **Circular wait**: there exists a set $\{P_0, P_1, \ldots, P_n\}$ of waiting processes such that P_0 is waiting for a resource that is held by P_1, P_1 is waiting for a resource that is held by P_2, …, P_{n-1} is waiting for a resource that is held by P_n, and P_n is waiting for a resource that is held by P_0.





Deadlock Prevention

Four necessary conditions must be satisfied **simultaneously** for deadlock to occur. Which of these can we remove?

- **Mutual exclusion:** only one process at a time can use a resource;
  Safety is an absolute requirement - removal cannot be used for prevention
- **Hold and wait:** a process holding at least one resource is waiting to acquire additional resources held by other processes;
- **No preemption:** a resource can be released only voluntarily by the process holding it, after that process has completed its task;
- **Circular wait:** there exists a set $\{P_0, P_1, \ldots, P_n\}$ of waiting processes such that P_0 is waiting for a resource that is held by P_1, P_1 is waiting for a resource that is held by P_2, …, P_{n-1} is waiting for a resource that is held by P_n, and P_n is waiting for a resource that is held by P_0.





Deadlock Prevention

Four necessary conditions must be satisfied **simultaneously** for deadlock to occur. Which of these can we remove?

- **Mutual exclusion:** only one process at a time can use a resource;
  - **Safety is an absolute requirement - removal cannot be used for prevention**
- **Hold and wait:** a process holding at least one resource is waiting to acquire additional resources held by other processes;
  - removal **impractical; Low resource utilization; starvation possible**
- **No preemption:** a resource can be released only voluntarily by the process holding it, after that process has completed its task;
- **Circular wait:** there exists a set $\{P_0, P_1, \ldots, P_n\}$ of waiting processes such that P_0 is waiting for a resource that is held by P_1, P_1 is waiting for a resource that is held by P_2, …, P_{n-1} is waiting for a resource that is held by P_n, and P_n is waiting for a resource that is held by P_0.





Deadlock Prevention

Four necessary conditions must be satisfied **simultaneously** for deadlock to occur. Which of these can we remove?

- **Mutual exclusion:** only one process at a time can use a resource;
  Safety is an absolute requirement - removal cannot be used for prevention
- **Hold and wait:** a process holding at least one resource is waiting to acquire additional resources held by other processes;
  - removal **impractical; Low resource utilization; starvation possible**
- **No preemption:** a resource can be released only voluntarily by the process holding it, after that process has completed its task;
  - removal impractical for most systems
- **Circular wait:** there exists a set $\{P_0, P_1, \ldots, P_n\}$ of waiting processes such that P_0 is waiting for a resource that is held by P_1, P_1 is waiting for a resource that is held by P_2, …, P_{n-1} is waiting for a resource that is held by P_n, and P_n is waiting for a resource that is held by P_0.





Deadlock Prevention

Four necessary conditions must be satisfied **simultaneously** for deadlock to occur. Which of these can we remove?

- **Mutual exclusion:** only one process at a time can use a resource;
  Safety is an absolute requirement - removal cannot be used for prevention
- **Hold and wait:** a process holding at least one resource is waiting to acquire additional resources held by other processes;
  - removal **impractical; Low resource utilization; starvation possible**
- **No preemption:** a resource can be released only voluntarily by the process holding it, after that process has completed its task;
  - removal impractical for most systems
- **Circular wait:** there exists a set $\{P_0, P_1, \ldots, P_n\}$ of waiting processes such that P_0 is waiting for a resource that is held by P_1, P_1 is waiting for a resource that is held by P_2, …, P_{n-1} is waiting for a resource that is held by P_n, and P_n is waiting for a resource that is held by P_0. **can be used in practice.**





Deadlock Prevention

Four necessary conditions must be satisfied **simultaneously** for deadlock to occur. Which of these can we remove?

- **Mutual exclusion:** only one process at a time can use a resource;
  Safety is an absolute requirement - removal cannot be used for prevention
- **Hold and wait:** a process holding at least one resource is waiting to acquire additional resources held by other processes;
  - removal **impractical; Low resource utilization; starvation possible**
- **No preemption:** a resource can be released only voluntarily by the process holding it, after that process has completed its task;
  - removal impractical for most systems
- **Circular wait:** there exists a set $\{P_0, P_1, \ldots, P_n\}$ of waiting processes such that P_0 is waiting for a resource that is held by P_1, P_1 is waiting for a resource that is held by P_2, …, P_{n-1} is waiting for a resource that is held by P_n, and P_n is waiting for a resource that is held by P_0. **can be used in practice.**

e.g. hash table with per-bucket locks and an operation changeKeys(item, k1, k2) to move an item from one bucket to another can avoid deadlock by always acquiring the lock for the lower-numbered bucket before the one for the higher-numbered bucket..





Deadlock Prevention

Four necessary conditions must be satisfied **simultaneously** for deadlock to occur. Which of these can we remove?

- Mutual exclusion: generally cannot remove
- Hold and wait: possible, but low utilisation / starvation risk
- No preemption: impractical for many resources
- Circular wait: often the most practical condition to break



Do Quiz 13

- Password: avoidance



Deadlock avoidance

Ensure that the system will ***never*** enter a deadlock state;

requires additional ***a priori*** information available on **possible resource requests.**

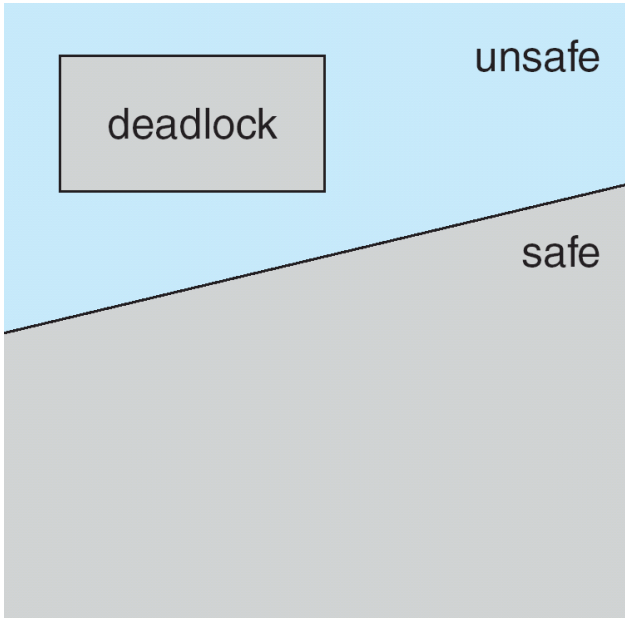
- Simplest and most useful model requires that each process declare the ***maximum number*** of resources of each type that it may need
- The deadlock-avoidance algorithm dynamically examines the resource-allocation state to ensure that there can **never be a circular-wait condition**
- Resource-allocation *state* is defined by the number of available and allocated resources, and the maximum demands of the processes

Deadlock avoidance: Safe State concept

System is in **safe state** if there **exists a sequence** $<P_1, P_2, \ldots, P_n>$ of ALL the processes in the system such that for each P_i the resources that P_i **can still request** can be satisfied by currently **available resources** + **resources held by all the** P_j, with $j < i$

That is:

- If P_i resource needs cannot be met now, then P_i can wait until all processes P_j ($j < i$) have finished executing.
  - When they have finished executing, they release all their resources and then P_i can obtain the resources needed , execute, return resources allocated, and terminate.
  - When P_i terminates, P_{i+1} can obtain its required resources, and so on .



- If a system is in **safe state** ⇒ **no deadlocks**
- If a system is in **unsafe state** ⇒ **possibility** of deadlock

Deadlock avoidance: Safe State concept

System is in **safe state** if

there **exists a sequence** $<P_1, P_2, \ldots, P_n>$ of ALL the processes in the system such that for each P_i

the resources that P_i **can still request** can be satisfied by currently **available resources** + **resources held by all the** P_j, with $j < I$

That is:

- If P_i resource needs cannot be met now, then P_i can wait until all processes P_j ($j < i$) have finished executing.
  - When they have finished executing they release all their resources and then P_i can obtain the resources needed, execute, return resources allocated, and terminate.
  - When P_i terminates, P_{i+1} can obtain its required resources, and so on

Consider a system with 12 resources and 3 **threads**: T_A, T_B, and T_C

at t_0 in **safe state:**

	Allocation	Max
T_A	5	10
T_B	2	4
T_C	2	9

at t_1 in **unsafe state** if:

	Allocation	Max
T_A	5	10
T_B	2	4
T_C	3	9

Deadlock avoidance: Safe State concept

System is in **safe state** if there **exists a sequence** $<P_1, P_2, \ldots, P_n>$ of ALL the processes in the system such that for each P_i the resources that P_i **can still request** can be satisfied by currently **available resources** + **resources held by all the** P_j, with $j < i$

That is:

- If P_i resource needs cannot be met now, then P_i can wait until all processes P_j ($j < i$) have finished executing.
  - When they have finished executing they release all their resources and then P_i can obtain the needed resources, execute, return allocated resources, and terminate
  - When P_i terminates, P_{i+1} can obtain its required resources, and so on

Consider a system with 12 resources and 3 **threads**:
$\mathbf{T_A}$, $\mathbf{T_B}$, and $\mathbf{T_C}$

at t_0 in **safe state:**

	Allocation	Max
$\mathbf{T_A}$	5	10
$\mathbf{T_B}$	2	4
$\mathbf{T_C}$	2	9

3 free

at t_1 in **unsafe state** if:

	Allocation	Max
$\mathbf{T_A}$	5	10
$\mathbf{T_B}$	2	4
$\mathbf{T_C}$	3	9

2 free

Deadlock avoidance: Safe State concept

Ensure that a system will **never** enter an **unsafe state**.

- Whenever a thread requests a resource that is currently available, the system decides whether the resource can be allocated immediately or the thread must wait.
  - The request is granted only if the allocation leaves the system in a **safe state**.
  - If a thread requests a resource currently available, may have to wait.
  - **Resource utilization may be lower** than it would otherwise be.

Consider a system with 12 resources and 3 **threads**:
T_A, T_B, and T_C

at t_0 in **safe state:**

	Allocation	Max
T_A	5	10
T_B	2	4
T_C	2	9

3 free

at t_1 in **unsafe state** if:

	Allocation	Max
T_A	5	10
T_B	2	4
T_C	3	9

2 free

T_C must wait



Deadlock avoidance algorithms

For:

- **single instance** of a resource type
  - Use a variant of the **resource-allocation graph**

- **multiple instances** of a resource type
  - Use the **Banker's Algorithm**

Deadlock avoidance: Resource-Allocation-Graph Algorithm

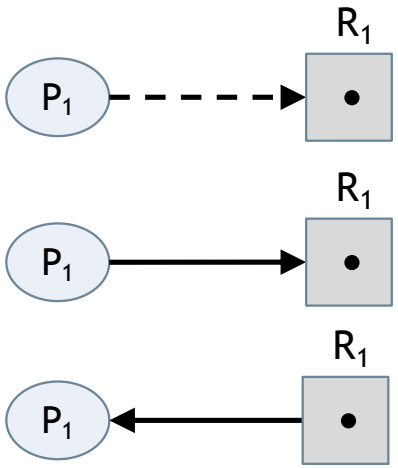
For **single instance** of a resource type.



Each process must *a priori* claim maximum resource use

Use a variant of the resource-allocation graph with *claim edges*.

- Claim edge $P_i \rightarrow R_j$ indicates that process P_j may request resource R_j; represented by a **dashed line**
- Claim edge **converts to request edge** when a process requests a resource
- **Request edge converted to an assignment edge** when the resource is allocated to the process
- When a resource is released by a process, assignment edge reconverts to a claim edge
- **A cycle in the graph implies** that the system is in **unsafe state**



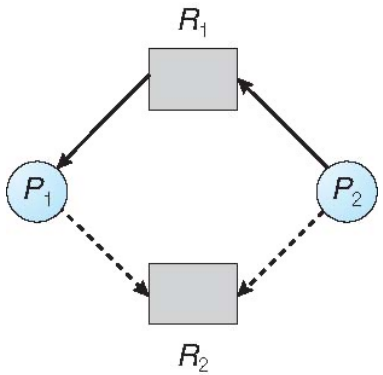
Resource-Allocation Graph Algorithm

Suppose that process P_i requests a resource R_j

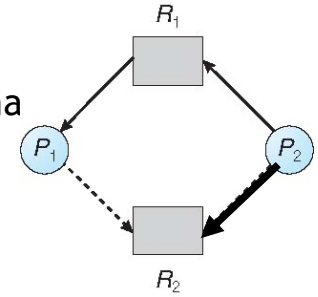
- The request **can be granted only** if converting the request edge to an assignment edge does **not result** in the formation of a **cycle** in the resource allocation graph
- Otherwise, the process must wait

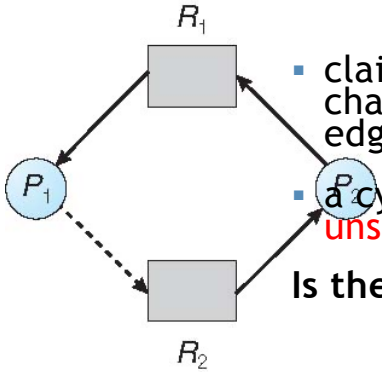
Check for safety with a **cycle-detection algorithm.**

- order n^2 operations, where **n** is the number of threads in the system.



- P1 is holding resource R1 and has a claim on R2
- P2 is requesting R1 and ha a claim on R2
- No cycle: a safe state.



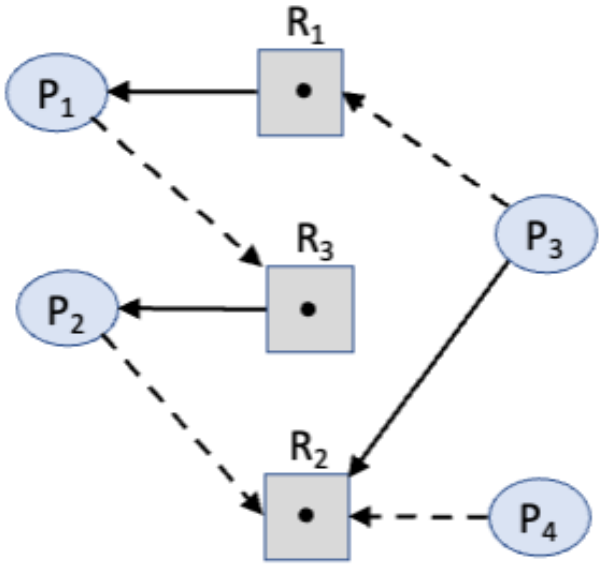


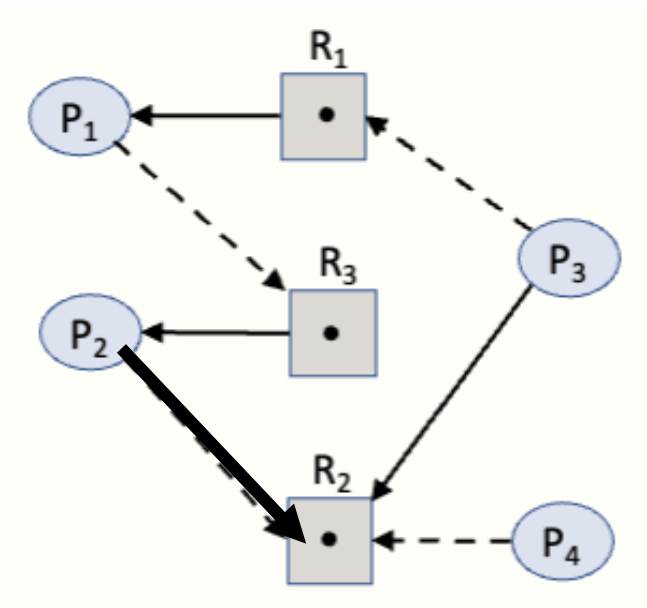
- claim edge from P2 to R2 changing to an assignment edge.
- a cycle in the graph ➔ unsafe state.

Is there a deadlock?

Deadlock avoidance – single resource instances

- In the **resource allocation graph** shown below, Process P_3 has requested a resource – should this request be granted? **Justify your answer.**

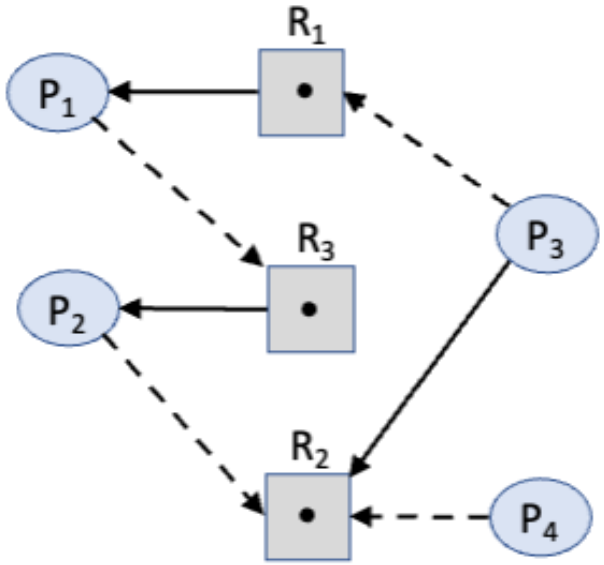




Deadlock avoidance – single resource instances

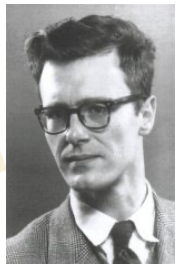
- In the **resource allocation graph** shown below, Process P_3 has requested a resource – should this request be granted? **Justify your answer.**

Suppose that process P_i requests a resource R_j
The request **can be granted only** if converting the request edge to an assignment edge does **not result** in the formation of a **cycle** in the resource allocation graph



Bankers Algorithm

Developed by Edsgar Dijkstra



*In the Banker's Algorithm, a thread states its **maximum resource requirements** when it begins a task, but it then acquires and releases those resources incrementally as the task runs.*
*The runtime system **delays granting some requests** to **ensure that the system never deadlocks**.*

Multiple instances of a resource type,

less efficient than resource-allocation graph.

- Each process must *a priori* **claim maximum use.**
- When a process requests a resource it may have to wait.
- When a process gets all its resources, it must return them in a finite amount of time.

Examples of resources with multiple instances:
- ***memory frames***
- ***Buffers in a buffer pool:*** *a collection of reusable fixed-size memory buffers used to temporarily hold data during input/output or communication.*
- ***Printers*** *in a printer pool*

Data structures (n – # threads, m- # resource types)

- **Available**: Vector of length *m*.
  - If available [*j*] = *k*, *k* instances of resource type R_j are available
- **Max**: *n* x *m* matrix.
  - If *Max* [*i,j*] = *k*, P_i may request at most *k* instances of resource type R_j
- **Allocation**: *n* x *m* matrix.
  - If Allocation[*i,j*] = *k* , P_i currently allocated *k* instances of R_j
- **Need**: *n* x *m* matrix.
  - If *Need*[*i,j*] = *k*, P_i may need at most *k* more instances of R_j to complete its task.
    - *Need* [*i,j*] = *Max*[*i,j*] – *Allocation* [*i,j*]

Bankers Algorithm: safety calculation

To calculate whether system is in a safe state

1. ***Work*** and ***Finish*** are vectors of length *m* and *n*, respectively. Initialize:
   Work = ***Available***
   Finish [*i*] = ***false*** **for** *i* = 0, 1, …, n - 1

2. Find an *i* such that both:
   (a) ***Finish*** [*i*] = ***false***
   (b) $Need_i \leq Work$

   If no such *i* exists, go to step 4

3. $Work = Work + Allocation_i$
   Finish[*i*] = ***true***
   go to step 2

4. If ***Finish*** [*i*] == ***true*** for all *i*, then the system is in a safe state.

   Otherwise, in an unsafe state.



Bankers Algorithm: safety calculation

To calculate whether system is in a safe state

1. ***Work*** and ***Finish*** are vectors of length *m* and *n*, respectively. Initialize:
   Work = ***Available***
   Finish [*i*] = ***false*** **for** *i* = 0, 1, …, n - 1

2. Find an *i* such that both:
   (a) ***Finish*** [*i*] = ***false***
   (b) $Need_i \leq Work$
   If no such *i* exists, go to step 4

3. ***Work*** = ***Work*** + $Allocation_i$
   Finish[*i*] = ***true***
   go to step 2

4. If ***Finish*** [*i*] == ***true*** for all *i*, then the system is in a safe state.
   Otherwise, in an unsafe state.

may require on order $m \times n^2$ operations to determine whether a state is safe.

Example: safety calculation

Consider a system with 3 resource types - *A* (10 instances), *B* (5), and *C* (7)
- and 5 **threads**: T_0, ...T_4

at t_0 :

	Allocation	Max	Available
	A B C	A B C	A B C
T_0	0 1 0	7 5 3	3 3 2
T_1	2 0 0	3 2 2	
T_2	3 0 2	9 0 2	
T_3	2 1 1	2 2 2	
T_4	0 0 2	4 3 3	

1. *Work* = *Available*
   Finish [*i*] = *false* for *i* = 0, 1, ..., n - 1
2. Find an *i* such that both:
   (a) *Finish* [*i*] = *false*
   (b) $Need_i \leq Work$
   If no such *i* exists, go to step 4
3. *Work* = *Work* + $Allocation_i$
   Finish[*i*] = *true*
   go to step 2
4. If *Finish* [*i*] == *true* for all *i*, then safe state. Otherwise, in an unsafe state.

Work	3 3 2
Finish	
0	false
1	false
2	false
3	false
4	false

Need* = *Max* - *Allocation

	Allocation	Need	Available
	A B C	A B C	A B C
T_0	0 1 0	7 4 3	3 3 2
T_1	2 0 0	1 2 2	
T_2	3 0 2	6 0 0	
T_3	2 1 1	0 1 1	
T_4	0 0 2	4 3 1	

Example: safety calculation

Consider a system with 3 resource types - *A* (10 instances), *B* (5), and *C* (7)
- and 5 **threads**: T_0, ...T_4

at t_0 :

	Allocation	Max	Available
	A B C	A B C	A B C
T_0	0 1 0	7 5 3	3 3 2
T_1	2 0 0	3 2 2	
T_2	3 0 2	9 0 2	
T_3	2 1 1	2 2 2	
T_4	0 0 2	4 3 3	

1. *Work* = *Available*
   Finish [*i*] = *false* for *i* = 0, 1, ..., n - 1
2. Find an *i* such that both:
   (a) *Finish* [*i*] = *false*
   (b) $Need_i \leq Work$
   If no such *i* exists, go to step 4
3. *Work* = *Work* + $Allocation_i$
   Finish[*i*] = *true*
   go to step 2
4. If *Finish* [*i*] == *true* for all *i*, then safe state. Otherwise, in an unsafe state.

Work	3 3 2
Finish	
0	false
1	false
2	false
3	false
4	false

Need* = *Max* - *Allocation

at t_0 :

	Allocation	Need	Available
	A B C	A B C	A B C
T_0	0 1 0	7 4 3	3 3 2
T_1	2 0 0	1 2 2	
T_2	3 0 2	6 0 0	
T_3	2 1 1	0 1 1	
T_4	0 0 2	4 3 1	

Example: safety calculation

Consider a system with 3 resource types - *A* (10 instances), *B* (5), and *C* (7)
- and 5 **threads**: T_0, ...T_4

at t_0 :

	Allocation	Max	Available
	A B C	A B C	A B C
T_0	0 1 0	7 5 3	3 3 2
T_1	2 0 0	3 2 2	
T_2	3 0 2	9 0 2	
T_3	2 1 1	2 2 2	
T_4	0 0 2	4 3 3	

1. *Work* = *Available*
   Finish [*i*] = *false* for *i* = 0, 1, …, n - 1
2. Find an *i* such that both:
   (a) *Finish* [*i*] = *false*
   (b) $Need_i \leq Work$
   If no such *i* exists, go to step 4
3. *Work* = *Work* + $Allocation_i$
   Finish[*i*] = *true*
   go to step 2
4. If *Finish* [*i*] == *true* for all *i*, then safe state. Otherwise, in an unsafe state.

Work	5 3 2
Finish	
0	false
1	true
2	false
3	false
4	false

Need* = *Max* - *Allocation

at t_0 :

	Allocation	Need	Available
	A B C	A B C	A B C
T_0	0 1 0	7 4 3	3 3 2
T_1	2 0 0	1 2 2	
T_2	3 0 2	6 0 0	
T_3	2 1 1	0 1 1	
T_4	0 0 2	4 3 1	

Example: safety calculation

Consider a system with 3 resource types - *A* (10 instances), *B* (5), and *C* (7)
- and 5 **threads**: T_0, ...T_4

at t_0 :

	Allocation	Max	Available
	A B C	A B C	A B C
T_0	0 1 0	7 5 3	3 3 2
T_1	2 0 0	3 2 2	
T_2	3 0 2	9 0 2	
T_3	2 1 1	2 2 2	
T_4	0 0 2	4 3 3	

1. *Work* = *Available*
   Finish [*i*] = *false* for *i* = 0, 1, ..., n - 1
2. Find an *i* such that both:
   (a) *Finish* [*i*] = *false*
   (b) $Need_i \leq Work$
   If no such *i* exists, go to step 4
3. *Work* = *Work* + $Allocation_i$
   Finish[*i*] = *true*
   go to step 2
4. If *Finish* [*i*] == *true* for all *i*, then safe state. Otherwise, in an unsafe state.

Work	5 3 2
Finish	
0	false
1	true
2	false
3	false
4	false

Need* = *Max* - *Allocation

at t_0 :

	Allocation	Need	Available
	A B C	A B C	A B C
T_0	0 1 0	7 4 3	3 3 2
T_1	2 0 0	1 2 2	
T_2	3 0 2	6 0 0	
T_3	2 1 1	0 1 1	
T_4	0 0 2	4 3 1	

Example: safety calculation

Consider a system with 3 resource types - *A* (10 instances), *B* (5), and *C* (7)
- and 5 **threads**: T_0, ...T_4

at t_0 :

	Allocation	Max	Available
	A B C	A B C	A B C
T_0	0 1 0	7 5 3	3 3 2
T_1	2 0 0	3 2 2	
T_2	3 0 2	9 0 2	
T_3	2 1 1	2 2 2	
T_4	0 0 2	4 3 3	

1. *Work* = *Available*
   Finish [*i*] = *false* for *i* = 0, 1, …, n - 1
2. Find an *i* such that both:
   (a) *Finish* [*i*] = *false*
   (b) $Need_i \leq Work$
   If no such *i* exists, go to step 4
3. *Work* = *Work* + $Allocation_i$
   Finish[*i*] = *true*
   go to step 2
4. If *Finish* [*i*] == *true* for all *i*, then safe state. Otherwise, in an unsafe state.

Work	7 4 3
Finish	
0	false
1	true
2	false
3	true
4	false

Need* = *Max* - *Allocation

at t_0 :

	Allocation	Need	Available
	A B C	A B C	A B C
T_0	0 1 0	7 4 3	3 3 2
T_1	2 0 0	1 2 2	
T_2	3 0 2	6 0 0	
T_3	2 1 1	0 1 1	
T_4	0 0 2	4 3 1	

Example: safety calculation

Consider a system with 3 resource types - *A* (10 instances), *B* (5), and *C* (7)
- and 5 **threads**: T_0, ...T_4

1. *Work* = *Available*
   Finish [*i*] = *false* for *i* = 0, 1, …, n - 1
2. Find an *i* such that both:
   (a) *Finish* [*i*] = *false*
   (b) $Need_i \leq Work$
   If no such *i* exists, go to step 4
3. *Work* = *Work* + $Allocation_i$
   Finish[*i*] = *true*
   go to step 2
4. If *Finish* [*i*] == *true* for all *i*, then safe state. Otherwise, in an unsafe state.

Work 7 5 3
Finish
0 true
1 true
2 false
3 true
4 false

Need = *Max* - *Allocation*

at t_0 :

	Allocation	Max	Available
	A B C	A B C	A B C
T_0	0 1 0	7 5 3	3 3 2
T_1	2 0 0	3 2 2	
T_2	3 0 2	9 0 2	
T_3	2 1 1	2 2 2	
T_4	0 0 2	4 3 3	



at t_0 :

	Allocation	Need	Available
	A B C	A B C	A B C
T_0	0 1 0	7 4 3	3 3 2
T_1	2 0 0	1 2 2	
T_2	3 0 2	6 0 0	
T_3	2 1 1	0 1 1	
T_4	0 0 2	4 3 1	

Example: safety calculation

Consider a system with 3 resource types - *A* (10 instances), *B* (5), and *C* (7)
- and 5 **threads**: T_0, ...T_4

1. *Work* = *Available*
   Finish [*i*] = *false* for *i* = 0, 1, ..., n - 1
2. Find an *i* such that both:
   (a) *Finish* [*i*] = *false*
   (b) $Need_i \leq Work$
   If no such *i* exists, go to step 4
3. *Work* = *Work* + $Allocation_i$
   Finish[*i*] = *true*
   go to step 2
4. If *Finish* [*i*] == *true* for all *i*, then safe state. Otherwise, in an unsafe state.

Work	10 5 5
Finish	
0	true
1	true
2	true
3	true
4	false

at t_0 :

	Allocation	Max	Available
	A B C	A B C	A B C
T_0	0 1 0	7 5 3	3 3 2
T_1	2 0 0	3 2 2	
T_2	3 0 2	9 0 2	
T_3	2 1 1	2 2 2	
T_4	0 0 2	4 3 3	

Need* = *Max* - *Allocation

at t_0 :

	Allocation	Need	Available
	A B C	A B C	A B C
T_0	0 1 0	7 4 3	3 3 2
T_1	2 0 0	1 2 2	
T_2	3 0 2	6 0 0	
T_3	2 1 1	0 1 1	
T_4	0 0 2	4 3 1	

Example: safety calculation

Consider a system with 3 resource types - *A* (10 instances), *B* (5), and *C* (7)
- and 5 **threads**: T_0, ...T_4

at t_0 :

	Allocation	Max	Available
	A B C	A B C	A B C
T_0	0 1 0	7 5 3	3 3 2
T_1	2 0 0	3 2 2	
T_2	3 0 2	9 0 2	
T_3	2 1 1	2 2 2	
T_4	0 0 2	4 3 3	

1. *Work* = *Available*
   Finish [*i*] = *false* for *i* = 0, 1, …, n - 1
2. Find an *i* such that both:
   (a) *Finish* [*i*] = *false*
   (b) $Need_i \leq Work$
   If no such *i* exists, go to step 4
3. *Work* = *Work* + $Allocation_i$
   Finish[*i*] = *true*
   go to step 2
4. If *Finish* [*i*] == *true* for all *i*, then safe state. Otherwise, in an unsafe state.

Work	10 5 7
Finish	
0	true
1	true
2	true
3	true
4	true

Need* = *Max* - *Allocation

at t_0 :

	Allocation	Need	Available
	A B C	A B C	A B C
T_0	0 1 0	7 4 3	3 3 2
T_1	2 0 0	1 2 2	
T_2	3 0 2	6 0 0	
T_3	2 1 1	0 1 1	
T_4	0 0 2	4 3 1	

Example: safety calculation

Consider a system with 3 resource types - *A* (10 instances), *B* (5), and *C* (7)
- and 5 **threads**: T_0, ...T_4

at t_0 :

	Allocation	Max	Available
	A B C	A B C	A B C
T_0	0 1 0	7 5 3	3 3 2
T_1	2 0 0	3 2 2	
T_2	3 0 2	9 0 2	
T_3	2 1 1	2 2 2	
T_4	0 0 2	4 3 3	

1. *Work* = *Available*
   Finish [*i*] = *false* for *i* = 0, 1, …, n - 1
2. Find an *i* such that both:
   (a) *Finish* [*i*] = *false*
   (b) $Need_i \leq Work$
   If no such *i* exists, go to step 4
3. *Work* = *Work* + $Allocation_i$
   Finish[*i*] = *true*
   go to step 2
4. If *Finish* [*i*] == *true* for all *i*, then safe state. Otherwise, in an unsafe state.

Work	10 5 7
Finish	
0	true
1	true
2	true
3	true
4	true

Safe!

Need* = *Max* - *Allocation

at t_0 :

	Allocation	Need	Available
	A B C	A B C	A B C
T_0	0 1 0	7 4 3	3 3 2
T_1	2 0 0	1 2 2	
T_2	3 0 2	6 0 0	
T_3	2 1 1	0 1 1	
T_4	0 0 2	4 3 1	

The system is in a **safe state** as the sequence < T_1, T_3, T_0, T_2, T_4> satisfies safety criterion

	Allocation	Max
	A B C	A B C
T_0	1 1 0	5 1 2
T_1	1 1 0	2 2 2
T_2	2 0 1	4 2 1
T_3	1 1 1	2 1 1

1. *Work* = *Available*
   Finish [*i*] = *false* for *i* = 0, 1, …, n – 1
2. Find an *i* such that both:
   (a) *Finish* [*i*] = *false*
   (b) $Need_i \leq Work$
   If no such *i* exists, go to step 4
3. *Work* = *Work* + $Allocation_i$
   Finish[*i*] = *true*
   go to step 2
4. If *Finish* [*i*] == *true* for all *i*, then safe state. Otherwise, in an unsafe state.

Example exam question

A system has 4 threads executing and 3 resource types - **A** (**6** instances), **B** (**3**), and **C** (**2**). At a particular point in time, the resource allocation to the threads is as shown in the grid below.

- Is this system in **a safe state**?

Example exam question

4 threads executing and 3 resource types - **A** (**5** instances), **B** (**4**), and **C** (**3**). At a particular point in time, the resource allocation to the threads is as shown in the grid below.

- Is this system in **a safe state**?

	Allocation			Max		
	A	B	C	A	B	C
T0	1	1	1	2	2	2
T1	0	1	1	1	2	1
T2	2	0	0	4	4	2
T3	1	1	1	4	4	1

1. Work = Available
Finish [*i*] = *false* for *i* = 0, 1, …, n – 1
2. Find an *i* such that both:
(a) *Finish* [*i*] = *false*
(b) $Need_i \leq Work$
If no such *i* exists, go to step 4
3. $Work = Work + Allocation_i$
Finish[*i*] = *true*
go to step 2
4. If *Finish* [*i*] == *true* for all *i*, then safe state. Otherwise, in an unsafe state.

Example exam question

4 threads executing and 3 resource types - **A** (**5** instances), **B** (**4**), and **C** (**3**). At a particular point in time, the resource allocation to the threads is as shown in the grid below.

- Is this system in **a safe state**?

	Allocation			Max		
	A	B	C	A	B	C
T0	1	1	1	2	2	1
T1	0	1	1	1	2	1
T2	2	0	0	4	3	2
T3	1	1	1	4	4	1

1. Work = Available
Finish [*i*] = *false* for *i* = 0, 1, …, n - 1
2. Find an *i* such that both:
(a) *Finish* [*i*] = *false*
(b) $Need_i \leq Work$
If no such *i* exists, go to step 4
3. $Work = Work + Allocation_i$
Finish[*i*] = *true*
go to step 2
4. If *Finish* [*i*] == *true* for all *i*, then safe state. Otherwise, in an unsafe state.

Do Quiz 14

- Password: recovery



Bankers Algorithm: Resource-Request for Process P_i

Let $Request_i$[...] be the request vector for process P_i.

$Request_i[j] = k$. Process P_i wants k instances of resource type R_j

1. If $Request_i \leq Need_i$ go to step 2. Otherwise, raise error condition, since process has exceeded its maximum claim
2. If $Request_i \leq Available$, go to step 3. Otherwise P_i must wait, since resources are not available
3. Pretend to allocate requested resources to P_i by modifying the state as follows:

$$Available = Available - Request_i$$
$$Allocation_i = Allocation_i + Request_i$$
$$Need_i = Need_i - Request_i$$

- If safe $\Rightarrow$ the resources are allocated to P_i
- If unsafe $\Rightarrow$ P_i must wait, and the old resource-allocation state is restored

Example: T_1 Request (1,0,2)

1. If $Request_i \le Need_i$ go to step 2.
   Otherwise, raise error condition.
2. If $Request_i \le Available$, go to step 3.
   Otherwise P_i must wait
3. Pretend to allocate requested resources to P_i:
   $Available = Available - Request_i$
   $Allocation_i = Allocation_i + Request_i$
   $Need_i = Need_i - Request_i$
   - If safe ⇒ the resources are allocated to P_i
   - If unsafe ⇒ P_i must wait, and the old resource-allocation state is restored

Check that Request ≤ Available ((1,0,2) ≤ (3,3,2) ⇒ true

Need* = *Max* - *Allocation

at t_0 :

	Allocation	Max	Available
	A B C	A B C	A B C
T_0	0 1 0	7 5 3	3 3 2
T_1	2 0 0	3 2 2	
T_2	3 0 2	9 0 2	
T_3	2 1 1	2 2 2	
T_4	0 0 2	4 3 3	

at t_1 :

	Allocation	Need	Available
	A B C	A B C	A B C
T_0	0 1 0	7 4 3	3 3 2
T_1	**2 0 0**	**1 2 2**	
T_2	3 0 2	6 0 0	
T_3	2 1 1	0 1 1	
T_4	0 0 2	4 3 1	

The system is in a **safe state** since
the sequence < T_1, T_3, T_4, T_2, T_0> satisfies safety criterion

Example: T_1 Request (1,0,2)

1. If $Request_i \le Need_i$ go to step 2.
   Otherwise, raise error condition.
2. If $Request_i \le Available$, go to step 3.
   Otherwise P_i must wait
3. Pretend to allocate requested resources to P_i :
   $Available = Available - Request_i$
   $Allocation_i = Allocation_i + Request_i$
   $Need_i = Need_i - Request_i$
   - If safe ⇒ the resources are allocated to P_i
   - If unsafe ⇒ P_i must wait, and the old resource-allocation state is restored

Check that Request ≤ Available ((1,0,2) ≤ (3,3,2) ⇒ true

Need* = *Max* - *Allocation

at t_0 :

	Allocation	Max	Available
	A B C	A B C	A B C
T_0	0 1 0	7 5 3	3 3 2
T_1	2 0 0	3 2 2	
T_2	3 0 2	9 0 2	
T_3	2 1 1	2 2 2	
T_4	0 0 2	4 3 3	

at t_1 :

	Allocation	Need	Available
	A B C	A B C	A B C
T_0	0 1 0	7 4 3	2 3 0
T_1	**3 0 2**	**0 2 0**	
T_2	3 0 2	6 0 0	
T_3	2 1 1	0 1 1	
T_4	0 0 2	4 3 1	

Example: T_1 Request (1,0,2)

1. If $Request_i \le Need_i$ go to step 2.
   Otherwise, raise error condition.
2. If $Request_i \le Available$, go to step 3.
   Otherwise P_i must wait
3. Pretend to allocate requested resources to P_i :
   $Available = Available - Request_i$
   $Allocation_i = Allocation_i + Request_i$
   $Need_i = Need_i - Request_i$
   - If safe ⇒ the resources are allocated to P_i
   - If unsafe ⇒ P_i must wait, and the old resource-allocation state is restored

Check that Request ≤ Available ((1,0,2) ≤ (3,3,2) ⇒ true

Need* = *Max* - *Allocation

at t_0 :

	Allocation	Max	Available
	A B C	A B C	A B C
T_0	0 1 0	7 5 3	3 3 2
T_1	2 0 0	3 2 2	
T_2	3 0 2	9 0 2	
T_3	2 1 1	2 2 2	
T_4	0 0 2	4 3 3	

at t_1 :

	Allocation	Need	Available
	A B C	A B C	A B C
T_0	0 1 0	7 4 3	**2 3 0**
T_1	**3 0 2**	**0 2 0**	
T_2	3 0 2	6 0 0	
T_3	2 1 1	0 1 1	
T_4	0 0 2	4 3 1	

The system is in a **safe state** since the sequence $< T_1, T_3, T_4, T_0, T_2 >$ satisfies safety criteria.
Request is therefore granted.

Example: T_1 Request (1,0,2)

1. If $Request_i \le Need_i$ go to step 2.
   Otherwise, raise error condition.
2. If $Request_i \le Available$, go to step 3.
   Otherwise P_i must wait
3. Pretend to allocate requested resources to P_i :
   $Available = Available - Request_i$
   $Allocation_i = Allocation_i + Request_i$
   $Need_i = Need_i - Request_i$
   - If safe ⇒ the resources are allocated to P_i
   - If unsafe ⇒ P_i must wait, and the old resource-allocation state is restored

Check that Request ≤ Available ((1,0,2) ≤ (3,3,2) ⇒ true

Need* = *Max* - *Allocation

at t_0 :

	Allocation	Max	Available
	A B C	A B C	A B C
T_0	0 1 0	7 5 3	3 3 2
T_1	2 0 0	3 2 2	
T_2	3 0 2	9 0 2	
T_3	2 1 1	2 2 2	
T_4	0 0 2	4 3 3	



at t_1 :

	Allocation	Need	Available
	A B C	A B C	A B C
T_0	0 1 0	7 4 3	**2 3 0**
T_1	**3 0 2**	**0 2 0**	
T_2	3 0 2	6 0 0	
T_3	2 1 1	0 1 1	
T_4	0 0 2	4 3 1	

Can request for (3,3,0) by T_4 now be granted?

Example: T_1 Request (1,0,2)

1. If $Request_i \le Need_i$ go to step 2.
   Otherwise, raise error condition.
2. If $Request_i \le Available$, go to step 3.
   Otherwise P_i must wait
3. Pretend to allocate requested resources to P_i :
   $Available = Available - Request_i$
   $Allocation_i = Allocation_i + Request_i$
   $Need_i = Need_i - Request_i$
   - If safe ⇒ the resources are allocated to P_i
   - If unsafe ⇒ P_i must wait, and the old resource-allocation state is restored

Check that Request ≤ Available ((1,0,2) ≤ (3,3,2) ⇒ true

Need* = *Max* - *Allocation

at t_0 :

	Allocation	Max	Available
	A B C	A B C	A B C
T_0	0 1 0	7 5 3	3 3 2
T_1	2 0 0	3 2 2	
T_2	3 0 2	9 0 2	
T_3	2 1 1	2 2 2	
T_4	0 0 2	4 3 3	

at t_1 :

	Allocation	Need	Available
	A B C	A B C	A B C
T_0	0 1 0	7 4 3	**2 3 0**
T_1	**3 0 2**	**0 2 0**	
T_2	3 0 2	6 0 0	
T_3	2 1 1	0 1 1	
T_4	0 0 2	4 3 1	

Can request for (0,2,0) by T_0 now be granted?

Another example

A system has 4 threads executing and 3 resource types - *A* (6 instances), *B* (3), and *C* (2). At a particular point in time, the resource allocation to the threads is as shown in the grid to the right.

- Using the Banker's Algorithm for deadlock avoidance, is this system in a safe state? Justify your answer, showing your calculations.

	Allocation	Max
	A B C	A B C
T_0	2 0 1	4 2 1
T_1	1 1 0	5 1 2
T_2	1 1 0	2 2 1
T_3	1 1 1	2 1 1

Another example

A system has 4 threads executing and 3 resource types: ***A* (3 instances)**, ***B* (3)**, and ***C* (4)**. At a particular point in time, it is in **a safe state**, with the resource allocation to the threads as shown in the grid below. Thread T_0 now requests one instance of resource B. In accordance with a policy of **deadlock avoidance**, should this request be granted? Show all your calculations.

	Allocation	Max
	A B C	A B C
T_0	1 0 1	1 1 1
T_1	1 0 1	2 0 1
T_2	1 1 0	3 1 2
T_3	0 1 1	2 3 3

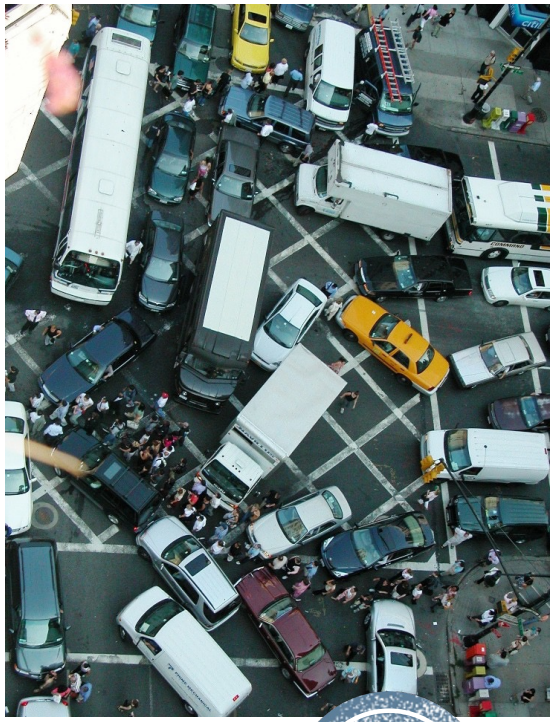
Handling Deadlock: Recovery from Deadlock

Michelle Kuttel

Department of Computer Science

University of Cape Town

June 2020





Deadlock recovery

Allow system to enter deadlocked state and then recover.

Requires:

- **Deadlock detection**

  Detection algorithms for
  - Single instance of a resource type
  - Multiple instances of a resource type.

- **Recovery scheme**

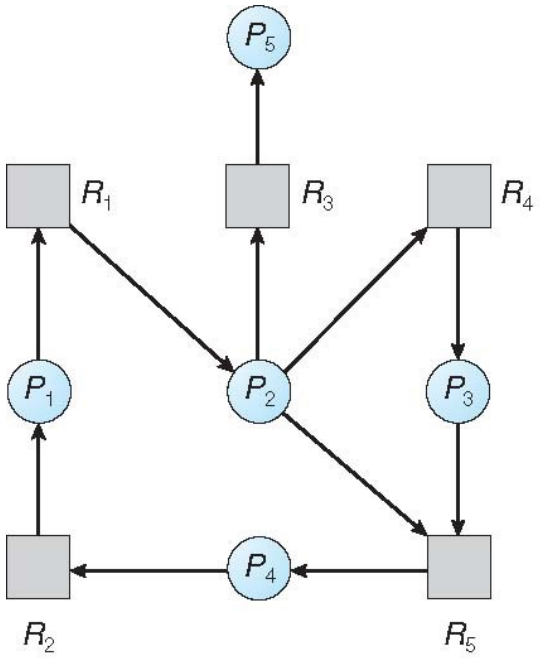
Deadlock detection: single instance of each resource type

Maintain a **wait-for** graph

- Nodes are processes
- $P_i \rightarrow P_j$ if P_i is waiting for P_j

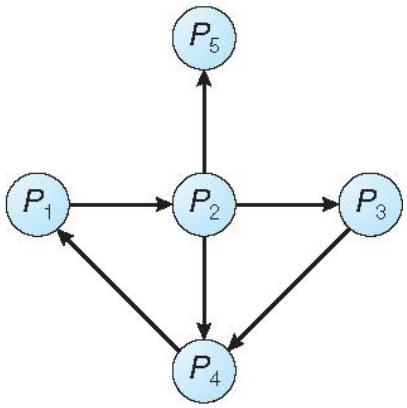
System needs to maintain the **wait-for** graph and periodically invoke an algorithm to search for a cycle in the graph.

- cycle -> deadlock
- n^2 operations, where n is the number of vertices in the graph



(a)

Resource-Allocation Graph



(b)

Corresponding wait-for graph



Deadlock detection: multiples instances of each resource type

Use algorithm similar to Banker's algorithm

- requires an order of m x n^2 operations to detect whether the system is in a deadlocked state.
  - m – resources
  - n - threads



Recovery from Deadlock: Process Termination

Options:

- Abort **all deadlocked processes**
- Abort one process at a time until the deadlock cycle is eliminated
  - In which order should we choose to abort?

Recovery from Deadlock: Resource Preemption



- **Selecting a victim** – minimize cost
- **Rollback** – return to some safe state, restart process for that state
- **Starvation** – same process may always be picked as victim, include number of rollback in cost factor – recovery can create fairness issues

Recovery from Deadlock: Proceed without resource



- e.g. Amazon web site

  if inventory server not available for a specific length of time, front end web server will give up, complete order and queue a background check, sending apology to customer if item not available. This is preferable to hanging indefinitely.



Recovery from Deadlock: Transactions

Transactions are used widely in file systems and databases.

Transaction either complete (and transaction commits) or does not happen at all (aborts).

Transactional systems are good for deadlock recovery:

- Thread roll back
  - Roll back or undo threads actions to a clean state (picking one or more victim threads)
- Thread restarting
  - Victim thread is restarted.



Java thread dumps

Java does not provide explicit support for deadlock detection.

A thread dump can be used to analyse a running program to determine if there is a deadlock.

- snapshot of the states of all threads in a Java application.
- show locking information, including which locks a blocked thread is waiting to acquire.

To generate a thread dump of a running application, from the command line enter:

- **jstack -l <pid>**

Deadlock overall

- Characterize with the four necessary conditions
- Prevent by breaking a condition
- Avoid by staying in safe states
- Detect with wait-for/resource graphs
- Recover by abort/rollback/restart
- In practice, many systems rely heavily on good design